Gestion de la dette technique

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Qu'est-ce que la Dette Technique et comment l'éviter au mieux **3**

A. Qu'est-ce que la dette technique 3

B. Comment la dette technique prend de l'ampleur ? 3

C. La dette technique est généralement prise en compte, mais pas comme il le faudrait 3

D. La dette technique est inévitable 3

E. Schéma vulgarisé de la dette technique 4

F. Comment identifier la dette technique ? 4

G. Comment réduire la dette technique ? 5

II. Exercice : Quiz **5**

III. Rembourser la dette technique **6**

A. Avant toute chose. Faire un état des choses. 6

B. Traiter intelligemment votre dette technique 6

C. Mettez en place du Refactoring 7

1. Quand savoir à quel moment il est nécessaire de refactorer ? 8
2. Est-ce qu'il ne faut que refactorer son code pour rembourser sa dette technique ? 8

D. Revoyez vos spécifications techniques 8

E. Mettez en place une Tierce Maintenance Applicative (TMA) 9

IV. Exercice : Quiz **9**

V. Essentiel **10**

VI. Auto-évaluation **10**

A. Exercice : 10

B. Test 11

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Qu'est-ce que la Dette Technique et comment l'éviter au mieux

Contexte

Bienvenue dans ce cours destiné à vous enseigner la gestion de la dette Technique, lorsque vous créez une solution, il est très rare que vous partiez d'une feuille réellement « *blanche* » vous réutilisez souvent des Frameworks ou des codes déjà créés que vous intégrez dans votre solution, l'objectif de ce cours est de vous faire prendre conscience de l'importance de la dette technique, mais surtout comment la maîtriser pour la rendre la moins lourde possible.

A. Qu'est-ce que la dette technique

Définition

Admettons que vous décidiez de mettre en place une solution Web, il est extrêmement rare de commencer avec une feuille blanche et que vous codiez la totalité de la solution sans faire usage de frameworks ou de librairies dans un certain langage. Parfois ces librairies sont faites par des tiers et il s'avère que le code ne soit pas aussi « propre » qu'il le devrait. Toutes ces erreurs créent ce qu'on appelle une dette technique qui faudra, à un moment donné, rembourser.

B. Comment la dette technique prend de l'ampleur ?

Comme expliqué, toutes les librairies et les frameworks que vous allez utiliser ne seront peut-être pas forcément développés comme il le faut. De ce fait, chaque défaut de code, librairie obsolète ou autre souci intentionnel ou non va créer de la dette technique. Cette dette technique se décompose en une qualité de code médiocre et donc une durée de vie inférieure à celle qui est donnée initialement.

Pour mieux comprendre la dette technique, comparez ça à une dette bancaire : sur une dette bancaire, vous empruntez de l'argent donc vous vous endettez, et vous rembourserez plus tard. Tandis que sur une dette technique, vous empruntez du temps de développement, qu'il faudra normalement rembourser en corrigeant toutes les anomalies créées.

C. La dette technique est généralement prise en compte, mais pas comme il le faudrait

Les agences de développement prennent généralement en compte la dette technique qu'ils appellent « *Provision pour risque* ». Cette provision pour risque prend en compte toutes les erreurs et les potentielles anomalies rencontrées par le code. Les agences facturent donc 20 % de temps supplémentaire pour réparer ces anomalies, cependant, même si cette dette technique est plus ou moins prise en compte, il est très fréquent qu'il faille, à un moment donné, rembourser plus que ce qu'il fallait.

D. La dette technique est inévitable

Il est important de préciser que la dette technique est inévitable, et cela est à imputer à la volonté même de gagner de temps. Lorsqu'une solution est en cours de développement, les développeurs souhaitent gagner du temps et gagner en productivité, cependant, les développeurs ont parfois du mal à déterminer si ce gain de temps et ce gain de productivité sera bénéfique à long terme. Nous sommes donc sur un élément extrêmement trompeur puisque gagner du temps à l'instant T, mais perdre deux fois plus de temps à l'instant T+1, est contre-productif.

Tous droits réservés à STUDI - Reproduction interdite

E. Schéma vulgarisé de la dette technique

Afin que vous visualisiez l'accumulation de la dette technique, voici un schéma vulgarisant cette dette :

F. Comment identifier la dette technique ?

Identifier une dette technique n'est pas une mince affaire, Bien qu'elle puisse se détecter par n'importe qui, il faut connaître les différents points alarmants permettant de détecter une dette technique élevée.

- Lorsque nous développons une solution, il n'est pas anodin que plusieurs personnes se développent en même temps. Cependant, si un membre d'une équipe indique qu'une seule et même personne est capable de modifier le code, cela est un indicateur inquiétant.
- Si le code n'est pas solide et qu'un développeur indique que si le code est modifié, il peut totalement cracher, cet indicateur est également générateur de dette technique, car si le développeur d'origine n'est plus sur le projet et qu'il est nécessaire de modifier le code, la dette technique générée serait énorme. Il faudrait tout reprendre depuis le début.
- En gestion de projet Scrum, les équipes ont une vélocité qui se traduit par la capacité à exécuter une charge de travail. Lorsque la vélocité baisse sans facteurs particuliers, cela indique que la dette technique grossit considérablement et qu'il est de plus en plus compliqué pour les développeurs de tenir la charge.
- Lorsque le projet est en production, si les anomalies généralisées se produisent de manière de répétée au point d'impacter la fiabilité du système, la dette technique est alors critique.
- Lorsque le projet est en développement, il n'est pas impossible que les développeurs emploient les mauvais choix, même si le design est bien exécuté, il se peut parfois que toutes ces erreurs ne soient pas détectées directement. La dette technique sera identifiée différemment. Nous la rencontrerons lors des releases de chaque version et toutes les anomalies détectées. Ces anomalies feront également baisser la vélocité de l'équipe.

Tous droits réservés à STUDI - Reproduction interdite

Ces indicateurs annoncent de dangereuses nouvelles et il sera impératif, soit de les régler, soit de rembourser cette dette technique. Il faut donc prendre le sujet à bras le corps et traiter le plus rapidement possible ce type d'anomalies pour ne pas se retrouver à un point de non-retour.

G. Comment réduire la dette technique ?

Fondamental

Même si la dette technique est inévitable, il est possible de la maîtriser et la réduire au mieux.

- Dans un premier temps, il est important de bien choisir le langage de programmation qui va être utilisé dans la solution. Il est important de ne pas se laisser submerger par la mode. Si une agence maîtrise un langage à la perfection et que ce langage répond à tous les besoins. Il n'est pas nécessaire de prendre un autre langage parce qu'il est « *cool* » et qu'il est en vogue.
- Bien qu'il ne faille pas choisir de langage de programmation en vogue, il en est de même pour les différents langages. Il est important de minimiser le nombre de technologies utilisées, plus les technologies sont maîtrisées, maintenues à jour et fiabilisées, plus vous diminuez votre risque de dette technique. Encore une fois, prendre une nouvelle technologie qui n'a pas fait ses preuves, peu connue et qui surtout, est encore dans une version faible, a pour effet de complexifier la maintenance et par conséquent la dette technologique.
- Mettez en place des tests unitaires, ces tests sont à mettre en place en même temps que votre code, au moment où vous terminez une phase de votre code, testez-le pour vous assurer qu'il répond correctement aux stimuli que vous avez mis en place. En mettant ces tests unitaires, vous limitez les effets tunnels et empêchez une régression de votre code, car il est testé, fiable et fonctionnel. Il n'y a ainsi aucune raison qu'il ne fonctionne plus à la livraison.
- Lorsque vous mettez votre code en place, respectez les règles, indentez bien votre code, commentez-le correctement, partagez vos méthodes avec vos équipes. Si vous sentez que cela est nécessaire, fonctionnez en pair programming cela évite les fautes et les connaissances du code se multiplient.
- Mettez en place de la documentation technique. Détaillez-là, et expliquez toutes les complexités ou toutes les subtilités pouvant survenir sur votre code pour qu'elles puissent être anticipés.

Exercice : Quiz

Question 1

La dette technique se rembourse :

- ○ En temps passé à corriger les erreurs.
- ○ Financièrement.

Question 2

Certains projets n'ont aucune dette technique.

- ○ Vrai
- ○ Faux

Question 3

Trouvez l'intrus :

- ○ 3 technologies utilisées.
- ○ Nouveau langage de développement.
- ○ Mise en place de Pair Programming.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

À quel moment se rembourse la dette technique ?

- Après le déploiement.
- Le plus tôt possible.
- Pendant le code.

Question 5

Qui est le principal fautif de la dette technique ?

- La malchance.
- Le Product Owner.
- Le développeur.
- Le client.

II. Rembourser la dette technique

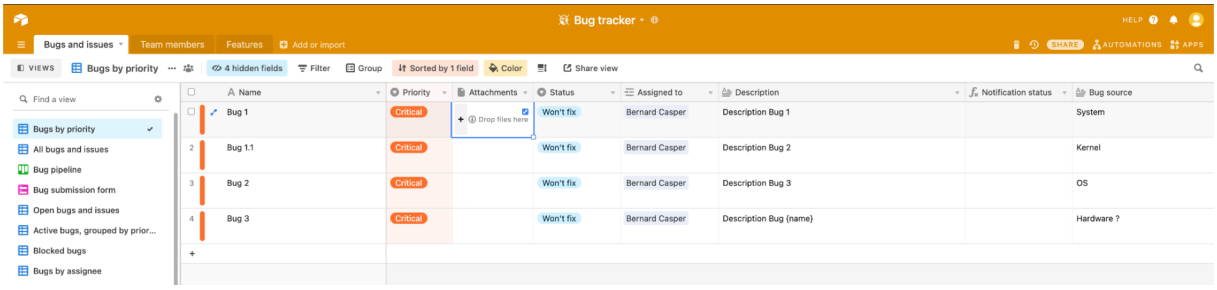
A. Avant toute chose. Faire un état des choses.

C'est un fait, les choses ont fait qu'une dette technique s'est accumulée et il est maintenant temps de la liquider. Il faut pour cela mettre en place différentes actions et surtout, repartir sur une bonne base. La première chose à faire est donc de mettre en place un audit pour déterminer toutes les problématiques, anomalies, bugs, etc.

Il est intéressant de pouvoir les répertorier dans une solution qui nous permettra de les traiter avec intelligence et non pas impulsivement au risque de créer davantage de problèmes.

Nous pouvons donc utiliser la solution **AirTable** qui se présente comme ceci en gestion de bugs :

AirTable, est une solution que l'on peut découvrir gratuitement, mais qui représente un investissement de 12 €/mois. Cependant, au vu de l'importance de cette solution, l'investissement est très maigre. En outre, il existe aussi d'autres solutions gratuites.



Dans cette capture d'écran, nous simulons des bugs répertoriés

Dans cette capture d'écran, nous avons donc la liste de bugs qui ont été répertoriés. Il faut maintenant les traiter.

B. Traiter intelligemment votre dette technique

Tout est au point, nous savons quelles sont les anomalies sur notre solution. L'erreur à ne pas faire est bien évidemment de se lancer dans une semaine intense de correction.

En effet, vous risquez un épuisement intellectuel, psychologique et physique précoce.

Priorisez ainsi l'itération.

Tous droits réservés à STUDI - Reproduction interdite

Définition

Lorsque nous parlons d'Itération ou de Sprint, il s'agit d'une période déterminée dans le temps durant laquelle vous vous concentrez exclusivement sur une tâche tel un sprint lors d'une course, vous donnez votre maximum pour atteindre vos objectifs dans le temps imparti.

Votre itération doit être totalement mesurée. Vous devez prendre un petit morceau de code, l'améliorer, l'optimiser le plus possible, sans excès de zèle pour ne pas produire davantage de dégâts.

C. Mettez en place du Refactoring

Définition

Le refactoring est une opération destinée à revoir le code source d'une solution digitale. L'idée n'est pas forcément de rajouter des fonctionnalités ni de corriger des bogues, mais surtout d'améliorer la « *beauté* » du code, améliorer la lisibilité et par conséquent, la maintenance.

Au moment où vous avez créé votre code, vous avez probablement mis en place du *bad code* et cela n'est pas entièrement votre faute dans le sens ou vous souhaitiez bien faire en :

- Gagnant un maximum de temps,
- En essayant de faire du code qui soit rapide et fonctionnel sans prendre le temps de se demander s'il sera optimal.

De plus, le métier de développeur exigent une grande concentration ce qui entraîne nécessairement de la fatigue comme tout un chacun.

Cependant, ce *bad code* ne va pas du tout vous aider, car c'est tous ces éléments qui ont créé votre dette technique.

Maintenant que vous avez pris conscience des conséquences néfastes d'un manque de soin de son code, essayons de transformer ce *bad code* en *good code*.

Ce *good code* va être l'antipode de votre *bad code* vous l'avez compris :

- Le code est indenté comme il faut,
- Les commentaires sont mis à bon escient,
- Les opérations sont bien nommées et permettent de se retrouver,
- Le code est testé, fiabilisé,
- Le code est compréhensible par tous les développeurs appliquant ce langage,
- Le code possède un minimum de dépendances,
- Les classes et les méthodes sont bien déclarées et exécutent ce dont pourquoi elles ont été créées.

Vous vous direz à juste titre « *pourquoi faire un code dans les règles de l'art alors que mon code fonctionne tout aussi bien ?* » et vous tomberez dans le piège de la dette technique.

Appliquer ce code propre limite votre dette technique. Faire du *bad code* va vous faire perdre du temps et vous devrez refactorer votre code.

En refactorant votre code, vous rembourserez donc une partie de votre dette technique.

Tous droits réservés à STUDI - Reproduction interdite

1. Quand savoir à quel moment il est nécessaire de refactorer ?

Lorsque vous êtes amenés à repasser plusieurs fois sur votre code, il est temps de penser que quelque chose ne va pas. Et c'est à ce moment-là qu'il faut également commencer à se poser des questions.

Nous pourrions trouver également de très nombreux facteurs nous indiquant qu'il est impératif de refactorer son code comme :

- Lorsqu'on se rend compte qu'on ne comprend même plus son code,
- Lorsqu'on veut ajouter une nouvelle fonctionnalité, mais qu'il n'est pas possible de modifier le code sans pour autant risquer d'endommager totalement la solution,
- Quand la stack technique utilisée n'est pas en adéquation avec vos compétences réelles et qu'il est nécessaire de revenir à des basiques.

2. Est-ce qu'il ne faut que refactorer son code pour rembourser sa dette technique ?

Alors, oui, mais non, sinon ce serait trop simple, le refactoring comme il est expliqué ici paraît simple, mais en réalité est extrêmement complexe à mettre en place, car il faut reprendre la totalité du code et le remettre dans un gabarit respectable, il faut donc se replonger au moment où vous avez créé le code original. Ensuite, il faut revoir toutes les anomalies et reprendre le code correctement.

Donc il est bien vrai que refactorer son code rembourse une bonne partie de la dette technique, mais comme nous l'avons expliqué, le refactoring ne corrige pas les bugs. Il faudra donc ensuite revoir tous les bogues pour les corriger.

Au-delà du refactoring, il sera important de vous imposer des tests unitaires pour vous assurer que votre code est fiable et fonctionnel.

D. Revoyez vos spécifications techniques

Comme nous l'avons expliqué plus haut, une des erreurs à ne pas commettre consiste à mettre en place trop de technologies en même temps pour éviter les incompatibilités. Mais si nous en arrivons à lire ça, c'est que l'erreur est faite et qu'il est maintenant temps de faire des choix et choisir les technologies nécessaires pour rembourser votre dette technique.

La question à vous poser à ce moment est exactement la même que celle qui faut se poser pour limiter la dette technique sauf que l'approche est différente cette fois.

Nous ne devons pas nous demander combien de temps nous allons économiser, mais combien de temps allons-nous mettre pour corriger toutes les anomalies, et faire passer notre fonctionnalité totalement instable en une fonctionnalité fiable, performante, et bien codée.

Il est donc important de prendre les technologies, librairies, frameworks, etc. que vous connaissez et surtout que vous maîtrisez.

Si vous ne maîtrisez pas une technologie, choisissez une autre. Il n'est pas question de perdre encore plus de temps, et encore moins de grossir votre dette technique. Parce que prendre une technologie que vous ne connaissez pas est facteur d'encore plus d'erreurs.

Tous droits réservés à STUDI - Reproduction interdite

E. Mettez en place une Tierce Maintenance Applicative (TMA)

Une fois qu'une solution est mise en place. Il est important de la maintenir. C'est à ce moment que la Tierce Maintenance Applicative (TMA) entre en jeu.

La TMA peut jouer sur différents terrains :

- **TMA Corrective**

  La TMA Corrective permet de corriger toutes les anomalies recensées par les utilisateurs, elle n'a que pour objectif de corriger.

- **TMA Évolutive**

  La TMA Évolutive permet de faire évoluer la solution. L'évolution de la solution peut se traduire par une diminution de la dette technique, de refactoring, etc.

La TMA est un élément permettant de ne pas vous laisser submerger par la dette technique, elle va vous aider à ne pas vous retrouver avec des équipes aux vélocités négatives.

Conclusion Générale

En conclusion, la dette technique est inévitable, mais elle est causée par l'envie viscérale de diminuer les temps de développement, optimiser la productivité à mauvais escient et en utilisant du code qui n'est pas compatible, obsolète ou alors mal développé à l'origine sur une solution qui est basée sur des concepts « *propres* ». Bien que la création de la dette technique soit impossible à éviter, il est possible de la limiter considérablement en respectant des règles essentielles.

Si par malheur la dette technique devenait trop conséquente au point d'impacter la productivité de la solution, il est tout de même possible de la rembourser, cependant, le prix à payer peut parfois être très lourd de conséquences, qu'elles soient humaines, financières ou relationnelles avec vos clients.

Un conseil, mettez en place du code propre dès le début, vos démarches de remboursement de dettes technique ne se feront même pas sentir, car elles seront minimes en comparaison avec ce que vous devrez mettre en place si rien n'est correctement respecté.

Exercice : Quiz

Question 1

En quoi consiste le refactoring ?

- ○ La refonte totale du code.
- ○ L'ajout de nouvelles fonctionnalités.

Question 2

La dette technique ne se rembourse qu'avec du refactoring.

- ○ Vrai
- ○ Faux

Question 3

Que signifie TMA ?

- ○ Tierce Maintenance Applicative
- ○ Tierce Maintenance d'Application
- ○ Triple Maintenance Applicative

Tous droits réservés à STUDI - Reproduction interdite

- Totale Maintenance d'Application

Question 4

Il est conseillé de traiter la dette technique en une seule fois.

- Vrai
- Faux

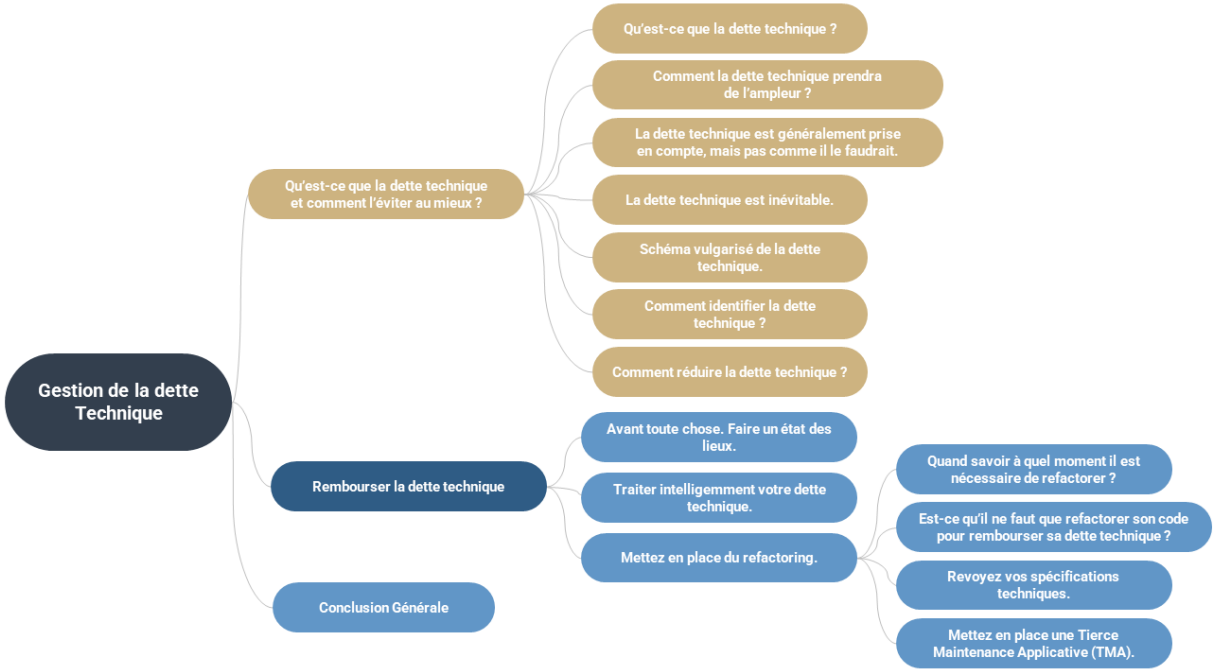
Question 5

À quoi va servir de répertorier toutes les erreurs sur une solution ?

- Cela va permettre de déterminer l'ampleur de la dette à rembourser.
- Cela va permettre de déterminer la fiabilité de l'application.

III. Essentiel

Dans ce cours, nous avons parlé exclusivement de la dette technique et de la négligence autour de celle-ci. De plus, nous avons parlé des différentes conséquences liées à cette négligence. Il est important de la limiter le plus possible en partant d'éléments simples, qui peuvent prendre un peu plus de temps au début, mais qui vous en fera gagner considérablement lorsque la solution sera en production et ne nécessite pas de repasser sur le code. En revanche, si la dette technique est malheureusement trop élevée, nous avons mis en place les différents éléments permettant de la rembourser le mieux possible.



IV. Auto-évaluation

A. Exercice :

Vous êtes arrivé en tant que Product manager dans une agence de développement d'applications.

Votre première mission est de faire un audit des différents projets sortant de votre agence.

Les résultats sont extrêmement inquiétants et les dettes techniques sont extrêmement élevées. Vous décidez donc de mettre en place une réunion avec tous les développeurs.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Quels sont vos conseils pour éviter d'accumuler votre dette technique ?

Question 2

Vous vous rendez compte que la dette technique d'un projet est considérable. Quelles sont vos préconisations d'urgence pour rembourser cette dette technique dans des délais optimaux ?

B. Test

Exercice

Trouver des solutions palliant aux problèmes énoncés.

Créer de la documentation technique

TMA

AirTable

Refactoring total

Choisir une technologie claire et connue

Trop de bad code	Besoin de corrections et d'améliorations	Besoin de répertorier les bugs	Utilisation du JS, React, Node, Angular	Aucune connaissance du code

Exercice : Quiz

Question 1

À quoi sert la provision pour risque ?

- Elle sert à anticiper les risques de retard.
- Elle sert à anticiper les risques d'absence du réalisateur du projet.
- Elle sert à anticiper les risques de non-paiement.

Question 2

En règle générale, les entreprises facturent combien de temps en plus de leur prestation initiale pour éviter de cumuler de la dette technique ?

- 15 %
- 20 %
- 25 %

Question 3

Que signifie TMA ?

- Triple Maintenance d'Application
- Tierce Maintenance d'Application
- Triple Maintenance Applicative

Tous droits réservés à STUDI - Reproduction interdite

- Tierce Maintenance Applicative

Question 4

La dette technique est inévitable.

- Faux
- Vrai

Question 5

Que permet AirTable ?

- De gérer ses tâches de la journée.
- De répertorier les bogues et collaborer dessus.
- De gérer un projet.

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 Solution n°1

Question 1

La dette technique se rembourse :

- (●) En temps passé à corriger les erreurs.
- ( ) Financièrement.

En principe, la dette technique se rembourse en temps passé à corriger les erreurs, cependant, le temps, c'est de l'argent donc les développeurs paient indirectement leurs dettes.

Question 2

Certains projets n'ont aucune dette technique.

- ( ) Vrai
- (●) Faux

C'est faux, à un moment donné, tous les projets ont une dette technique à rembourser.

Question 3

Trouvez l'intrus :

- ( ) 3 technologies utilisées.
- ( ) Nouveau langage de développement.
- (●) Mise en place de Pair Programming.

La mise en place de pair programming est l'intrus. Les deux autres sont des vecteurs augmentant la dette technique.

Question 4

À quel moment se rembourse la dette technique ?

- ( ) Après le déploiement.
- (●) Le plus tôt possible.
- ( ) Pendant le code.

La dette technique doit se rembourser le plus tôt possible.

Question 5

Qui est le principal fautif de la dette technique ?

- ( ) La malchance.
- ( ) Le Product Owner.
- (●) Le développeur.
- ( ) Le client.

Généralement et malheureusement, le principal fautif de la dette technique est le développeur qui ne respecte pas les règles principales d'un code optimal.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 9 Solution n°2

Question 1

En quoi consiste le refactoring ?

- ⦿ La refonte totale du code.
- ○ L'ajout de nouvelles fonctionnalités.

Q Le refactoring consiste à refonder totalement le code.

Question 2

La dette technique ne se rembourse qu'avec du refactoring.

- ○ Vrai
- ⦿ Faux

Q Faux, le refactoring ne sert qu'à remettre en place un code propre et compréhensible de tous, il ne corrige aucun bug.

Question 3

Que signifie TMA ?

- ⦿ Tierce Maintenance Applicative
- ○ Tierce Maintenance d'Application
- ○ Triple Maintenance Applicative
- ○ Totale Maintenance d'Application

Q TMA Signifie Tierce Maintenance Applicative.

Question 4

Il est conseillé de traiter la dette technique en une seule fois.

- ○ Vrai
- ⦿ Faux

Q Faux, il est conseillé de traiter la dette technique en itération. Pour éviter le surmenage et surtout un travail vain.

Question 5

À quoi va servir de répertorier toutes les erreurs sur une solution ?

- ⦿ Cela va permettre de déterminer l'ampleur de la dette à rembourser.
- ○ Cela va permettre de déterminer la fiabilité de l'application.

Q Répertorier toutes les erreurs en estimant le temps de réparation va permettre de déterminer l'ampleur de la dette à rembourser.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. Solution n°3

Il existe plusieurs façons pour limiter et pour éviter d'accumuler notre dette technique. Dans un premier temps, il est important de mettre en place un code propre, bien indenté, bien commenté pour que tout le monde puisse le comprendre. Il est également impératif de collaborer ; faire son code tout seul n'est pas du tout la meilleure façon de limiter la dette technique. De plus, n'utilisez pas de technologies que vous ne maîtrisez pas ou des nouvelles technologies n'ayant pas encore faites leurs preuves. Utilisez des technologies, des langages de programmation que vous maîtrisez.

Exercice p. Solution n°4

Pour rembourser cette dette dans des délais optimaux, il est nécessaire d'employer des grands moyens, dans un premier temps il est nécessaire de faire un audit de toutes les erreurs répertoriées sur le projet. Il ne sera absolument pas anodin de trouver du *bad code*, ce *bad code* devra impérativement être refactorer afin qu'il puisse être plus compréhensible par tous les développeurs. Une fois le code reconstruit, il faudra mettre en place différents sprints sur les différentes fonctionnalités qui posent problème. Chaque fonctionnalité devra obligatoirement faire office de correctifs. Il est également possible de mettre en place une TMA corrective et Évolutive pour améliorer tous ces soucis et rembourser au mieux la dette technique.

Exercice p. 11 Solution n°5

Trouver des solutions palliant aux problèmes énoncés.

Trop de bad code	Besoin de corrections et d'améliorations	Besoin de répertorier les bugs	Utilisation du JS, React, Node, Angular	Aucune connaissance du code
Refactoring total	TMA	AirTable	Choisir une technologie claire et connue	Créer de la documentation technique

Problèmes	Solution
Trop de bad code	Refactoring total
Besoin de corrections et d'améliorations	TMA
Besoin de répertorier les bugs	AirTable
Utilisation du JS, React, Node, Angular	Choisir une technologie claire et connue
Aucune connaissance du code	Créer de la documentation technique

Exercice p. 11 Solution n°6

Question 1

À quoi sert la provision pour risque ?

- (●) Elle sert à anticiper les risques de retard.
- ( ) Elle sert à anticiper les risques d'absence du réalisateur du projet.
- ( ) Elle sert à anticiper les risques de non-paiement.

Tous droits réservés à STUDI - Reproduction interdite

Lorsque l'on parle de provision pour risque, on prend en compte les risques liés aux projets qui pourront directement ou indirectement provoquer du retard sur la livraison du projet.

Question 2

En règle générale, les entreprises facturent combien de temps en plus de leur prestation initiale pour éviter de cumuler de la dette technique ?

- ○ 15 %
- ◉ 20 %
- ○ 25 %

Les entreprises lissent 20 % supplémentaire au prix initial pour éviter de cumuler de la dette technique.

Question 3

Que signifie TMA ?

- ○ Triple Maintenance d'Application
- ○ Tierce Maintenance d'Application
- ○ Triple Maintenance Applicative
- ◉ Tierce Maintenance Applicative

La TMA signifie *Tierce Maintenance Applicative*.

Question 4

La dette technique est inévitable.

- ○ Faux
- ◉ Vrai

La dette technique est effectivement inévitable.

Question 5

Que permet AirTable ?

- ○ De gérer ses tâches de la journée.
- ◉ De répertorier les bogues et collaborer dessus.
- ○ De gérer un projet.

AirTable est une plateforme en SaaS permettant de répertorier les bogues et collaborer dessus.

Tous droits réservés à STUDI - Reproduction interdite